Artificial Intelligence & Machine Learning

Phase 3

WITH,

DR. SWATI VERMA

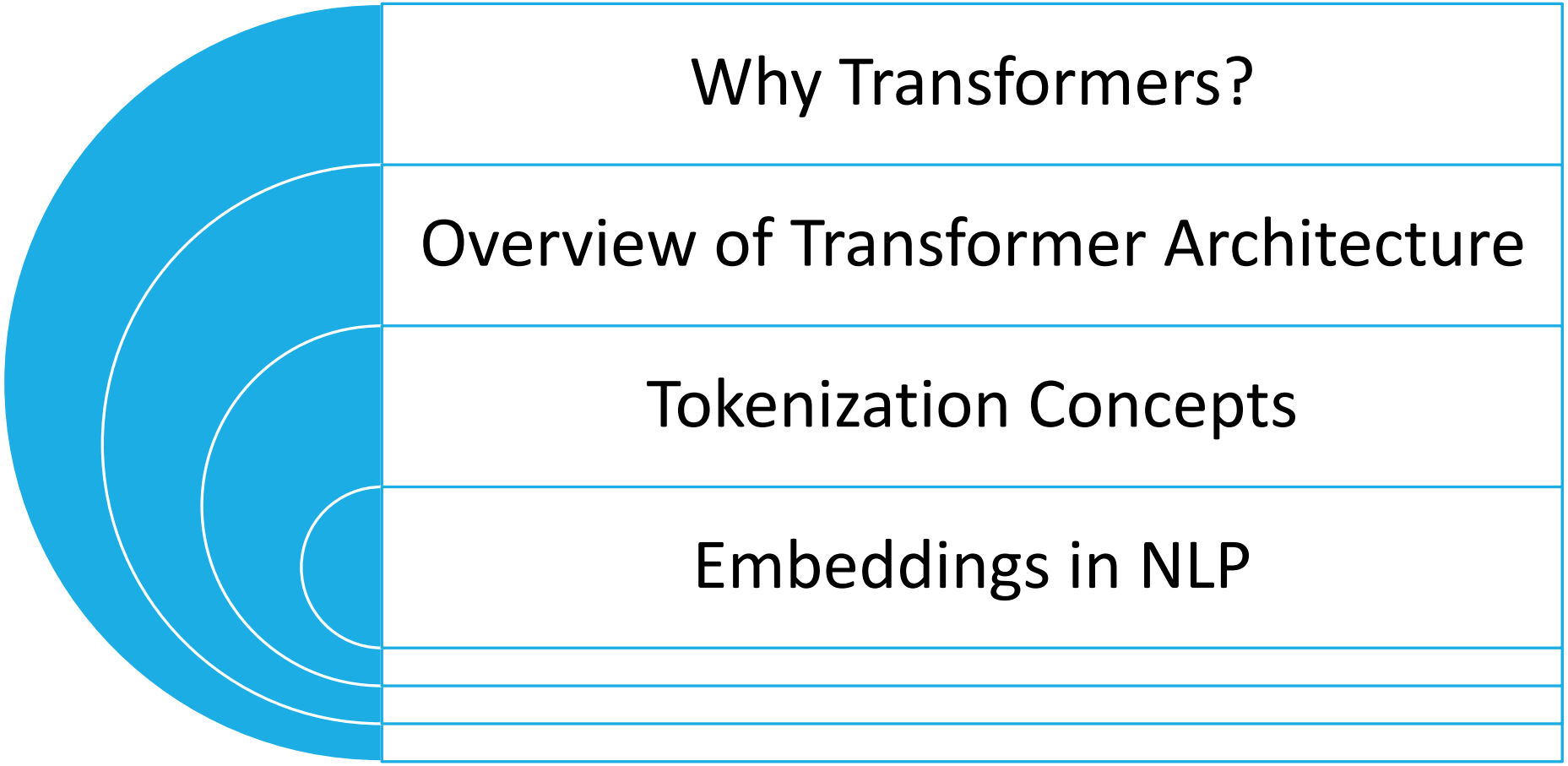
L&D CONSULTANT, MASTER TRAINER, CAPACITY ENHANCEMENT COACH

Program Structure Overview

Content:

- **Weekly Themes:**
  - Week 1 → Foundations & Neural Networks
  - Week 2 → CNNs & Optimization
  - Week 3 → Transformers & NLP
  - Week 4 → LangChain, RAG, Agents
  - Week 5 → API, Deployment, Capstone
- **Daily structure:**
  - **Theory (40%)**
  - **Hands-on Labs (60%)**

Week 3 - Learning Recap



Introduction to Large Language Models (LLMs)

WEEK 4 – DAY 1

Session Objectives

- Understand what **LLMs really are** (beyond buzzwords)
- Know **why "Large" matters**
- Understand **how LLMs are trained and evolved**
- Recognize **limitations, risks, and expectations**
- Be ready for **Prompt Engineering & RAG** in upcoming sessions

From NLP Models to LLMs

- **What we already know (Weeks 1–3)**
- Tokens
- Embeddings
- Attention
- Transformers
- NLP tasks (NER, QA, summarization, translation)

- **What changes with LLMs?**
- Scale
- Behavior
- Generalization
- Reasoning-like responses
- Enterprise risk

- **Key shift:**
  From *task-specific models → general-purpose language engines*

What Is a Large Language Model?

- A **Large Language Model** is:
  - A transformer-based neural network
  - Trained on **massive text corpora**
  - Capable of **predicting the next token**
  - Exhibits **emergent abilities** due to scale

- **Important**
- LLMs **do not understand meaning.**
- They learn **statistical patterns of language**

What Makes an LLM “Large”?

- **“Large” refers to:**

- **Training Data**
  - Books, articles, code, policies, forums
- **Number of Parameters**
  - Millions → Billions → Trillions
- **Training Compute**
  - GPUs / TPUs over weeks or months

- **Why it matters:**
  - Better generalization
  - More fluent language
  - Unexpected capabilities

Lab set-up

```
from transformers import pipeline

import textwrap


generator = pipeline(

    "text-generation",

    model="mistralai/Mistral-7B-Instruct-v0.1",

    max_new_tokens=200,

    temperature=0.7

)
```

```
def run_prompt(prompt):

    response =
generator(prompt)[0]["generated_text"]

    print("\nPROMPT:\n", prompt)

    print("\nRESPONSE:\n",
textwrap.fill(response, 100))
```

Why Lab Setup Exists Even Without Code

- Ensures consistent model behavior
- Removes tool-based variance
- Shifts focus from syntax to cognition
- Prepares for advanced prompt labs
- Mirrors real enterprise AI usage

LAB 1 – “What Is an LLM REALLY Doing?”

- **Objective**
- Break the myth that LLMs “understand”.

- **Task for Participants**
- Compare outputs
- Identify:
  - What changed?
  - Did knowledge change or only language style?

1. prompt_simple = "Explain what a transformer is in simple terms.“
2. prompt_child = "Explain what a transformer is to a 10-year-old child."
3. prompt_exec = "Explain what a transformer to a senior banking executive."

- **Learning**
  - Same knowledge
  - Different language shaping
  - No “understanding”, only probability-based adaptation

LAB 1 – “What Is an LLM REALLY Doing?”

Debrief (Key Insight)

- Same internal model
- No understanding change
- Only **probabilistic language shaping**

Why Scale Changes Behavior (Intuition)

- **Small Models:**
- Memorize patterns
- Perform narrow tasks
- Fail outside training scope

- **Large Models:**
- Generalize across domains
- Handle unseen prompts
- Appear to “reason”

- **Important:**
  Reasoning is **emergent**, not explicitly programmed

LAB 2 – Scale Changes Behavior (Emergent Ability)

- **Objective**
- Show why **"Large"** matters.
- **Activity -** Prompt:
  - "Write a professional email requesting leave."
- Then ask:
  - "Now rewrite it to follow Indian corporate HR tone."
- Then ask:
  - "Rewrite it as per BFSI compliance tone."

- **Thinking Task**
  - Was BFSI tone explicitly trained?
  - Where did this knowledge come from?

- **Key Learning**
  - Scale → latent patterns
  - No explicit BFSI rules stored
  - Emergent behavior

LLMs vs Traditional ML Models

Aspect	Traditional ML	NLP Models	LLMs
Task scope	Narrow	Task-specific	General
Training data	Structured	Domain text	Internet-scale
Output	Predictive	Task output	Free-form language
Risk	Low	Medium	High

Positioning Popular Models (High Level)

- **Encoder Models**
  - BERT
  - Used for: classification, NER, embeddings
- **Decoder Models**
  - GPT family
  - Used for: generation, chat, reasoning
- **Encoder-Decoder Models**
  - T5
- Used for: translation, summarization
- LLMs are **mostly decoder-only**

Why GPT-style Models Dominate LLMs

- **Decoder-only advantages:**
  - Natural text generation
  - Conversational flow
  - Autoregressive reasoning
  - Tool-calling friendly
- **Trade-off:**
  - Harder to control
  - Hallucination risk
  - Higher compute cost

LAB 3 – GPT vs BERT Thinking (Model Positioning)

- **Objective**
- To understand encoder vs decoder difference without code.
- **Activity**
- Perform two tasks:
- **Task A**
  - "Classify this sentence as HR / IT / Finance."
- **Task B**
  - "Write a response email to this sentence."
- **Discussion**
  - Which task feels natural?
  - Which feels forced?
- **Insight**
  - Encoder models → classification
  - Decoder models → generation
  - LLMs dominate generation-heavy workflows

How LLMs Are Trained (Conceptual)

- **Phase 1: Pre-Training**
  - Objective: predict next token
  - Massive unlabeled data
  - No task awareness
- **Phase 2: Fine-Tuning**
  - Domain-specific data
  - Instruction following
  - Safety alignment
- **Phase 3: Alignment**
  - Human feedback
  - Reinforcement Learning (RLHF)

What is RLHF

- **RLHF** stands for **Reinforcement Learning from Human Feedback**.

- It is a training approach used to align AI models—especially large language models—with human expectations, preferences, and real-world usage standards.

1. Why RLHF Is Needed

- Pretrained language models learn from large volumes of text using **unsupervised or self-supervised learning**. While they become good at predicting language, they may:
  - Produce unsafe or biased outputs
  - Ignore user intent
  - Provide technically correct but unusable answers
  - Respond in ways that are not aligned with business or ethical expectations
  - RLHF introduces **human judgment** to guide the model toward *preferred behavior*.

1. Why RLHF Is Needed

- **Step 1: Pretrained Base Model**
  - Model is trained on large text datasets
  - Learns grammar, facts, and general reasoning
- **Step 2: Human Feedback Collection**
  - Human reviewers evaluate model outputs
  - They **rank responses** (best → worst) or label quality
  - Criteria include:
    - Helpfulness
    - Correctness
    - Safety
    - Tone and clarity
- **Step 3: Reward Model Training**
  - A **reward model** is trained to predict human preferences
  - It assigns a score to each response based on how humans rated it
- **Step 4: Reinforcement Learning (Policy Optimization)**
  - The base model is fine-tuned using **reinforcement learning** (commonly PPO – Proximal Policy Optimization)
  - The model learns to maximize the reward score
  - Over time, outputs become more aligned with human expectations

RLHF vs Traditional Training

Aspect	Traditional ML Training	RLHF
Learning source	Static datasets	Human preferences
Feedback type	Correct / incorrect	Relative ranking
Focus	Accuracy	Alignment & usefulness
Adaptability	Limited	High

Where RLHF Is Used

- **Chatbots & Virtual Assistants**
- **Customer support automation**
- **Enterprise AI copilots**
- **Banking & BFSI applications** (tone, compliance, safe responses)
- **Content moderation systems**

- Given your work in **Data Analytics and AI training**, RLHF is especially relevant when explaining:
  - Why GenAI behaves “politely”
  - How AI learns organizational tone
  - Why AI avoids unsafe or non-compliant response

Pre-Training vs Fine-Tuning

Aspect	Pre-Training	Fine-Tuning
Data	Huge, generic	Small, targeted
Cost	Very high	Moderate
Purpose	Language understanding	Behavior shaping
Ownership	Model provider	Enterprise possible

LAB 4 – Pre-training vs Fine-tuning (Behavior Control)

- **Objective**
  - Understand why enterprises **do not trust base LLMs**.
- **Activity**
- Ask:
  - "Explain company leave policy."
- Then:
  - "Explain company leave policy assuming Indian labor law."
- Then:
  - "Explain company leave policy assuming US labor law."

- **Observation Task**
  - Did the model ask for clarification?
  - Did it assume facts?

- **Key Insight**
  - Pre-trained models guess
  - Fine-tuning & RAG reduce assumptions
  - Assumptions = compliance risk

Training & Evolution of LLMs

- **Evolution Path**
  - Static NLP models
  - Task-specific transformers
  - Instruction-tuned LLMs
  - Multi-modal LLMs
  - Tool-enabled agents

- Evolution is driven by **scale + feedback**

What LLMs Are Good At

- ✓ Language generation
- ✓ Summarization
- ✓ Question answering
- ✓ Pattern explanation
- ✓ Drafting & ideation

What LLMs Are NOT Good At

- ✗ Ground truth verification
- ✗ Guaranteed factual accuracy
- ✗ Real-time knowledge
- ✗ Logical consistency over long chains
- ✗ Regulatory decision-making alone

Why LLMs Hallucinate

- **Hallucination occurs because:**
  - LLMs predict **plausible text**
  - Not **true text**
  - No internal fact checker
  - Over-confidence when context is missing

- LLMs optimize **fluency**, not **truth**

When Hallucinations Increase

- Vague prompts
- Missing context
- Over-general questions
- Long generation chains
- Domain-specific compliance content

LAB 5 – Hallucination Trigger Lab

- **Objective**
- Let *intentionally cause hallucination*.
- **Prompt**
  - “Explain SLK internal HR escalation matrix.”
  - (There is none.)
- **Follow-up Prompt**
  - “Are you sure? Please confirm.”
- **Task**
  - Identify hallucinated elements
  - Highlight confidence level
- **Debrief**
  - Model optimized for fluency
  - Confidence ≠ correctness
- **Key line:**
  - “Hallucination is not a bug. It is a design outcome.”

Why Prompts Matter So Much

- Prompt = **Interface to the model's behavior**
- **Prompt controls:**
  - Tone
  - Detail level
  - Risk
  - Compliance
  - Reasoning depth

- Prompting is **not input**
  It is **instruction**

LAB 6 – Prompt Sensitivity Lab

- **Objective**
- Show prompt = behavior control.
- **Prompt Set**
- Give same content with different instructions:
  - “Summarize this policy.”
  - “Summarize this policy conservatively.”
  - “Summarize this policy for legal compliance.”
  - “Summarize this policy and highlight risks.”
- **Task**
- Compare:
  - Tone
  - Risk
  - Information loss
- **Learning**
  - Prompt is **not input**
  - Prompt is **instruction**

LLM Expectations vs Reality

- LLMs are used with:
  - Retrieval (RAG)
  - Tools
  - Human approvals
  - Monitoring
  - Governance layers

- We will build this architecture in coming sessions

LAB 7 – Expectation vs Reality Debate

- **Group A:**
  Argue: “LLMs can replace analysts.”
- **Group B:**
  Argue: “LLMs must always have humans.”

- **Rules**
- Use outputs from earlier labs as evidence
- No opinions without examples

LAB 8 – Mini Architecture Thought Lab

- **Scenario**
- "HR wants an LLM-based policy assistant."

- **Ask is:**
  - Can we connect LLM directly to users?
  - What could go wrong?

Summary

- What LLMs really are
- Why scale changes behavior
- How LLMs are trained
- Why hallucinations happen
- Why prompts matter

Discussion Questions

- Should LLMs be allowed to make decisions without humans?
- Is fluency more dangerous than wrong answers?
- Where would you *not* deploy an LLM?

Prompt Engineering & Advanced Prompting Techniques

WEEK 4 – DAY 2

Session Objectives

- Understand **why prompts control LLM behavior**
- Design **effective prompts** for business use cases
- Apply **advanced prompting patterns**
- Identify **prompt failures & hallucination risks**
- Use prompts for **analysis, reasoning, and evaluation**

SECTION 1: WHY PROMPT ENGINEERING MATTERS

Why Prompts Are Critical

- LLMs do **not think**
- They **predict next tokens** based on instructions + context
- Prompt = **Control Layer**
- Poor prompt → hallucination, vague output, wrong tone
- **Business Reality**
  - Same model
  - Same data
  - Different prompt
    → Completely different business outcome

LAB 2A – Prompt Sensitivity Experiment

- **Objective**
- Show how **small prompt changes** drastically affect outputs.

- **Instructions (No Code – Copilot / ChatGPT)**
- Ask participants to run:
  - Explain what a transformer is.
- Then:
  - Explain what a transformer is in simple terms.
- Then:
  - Explain what a transformer to a senior banking executive.

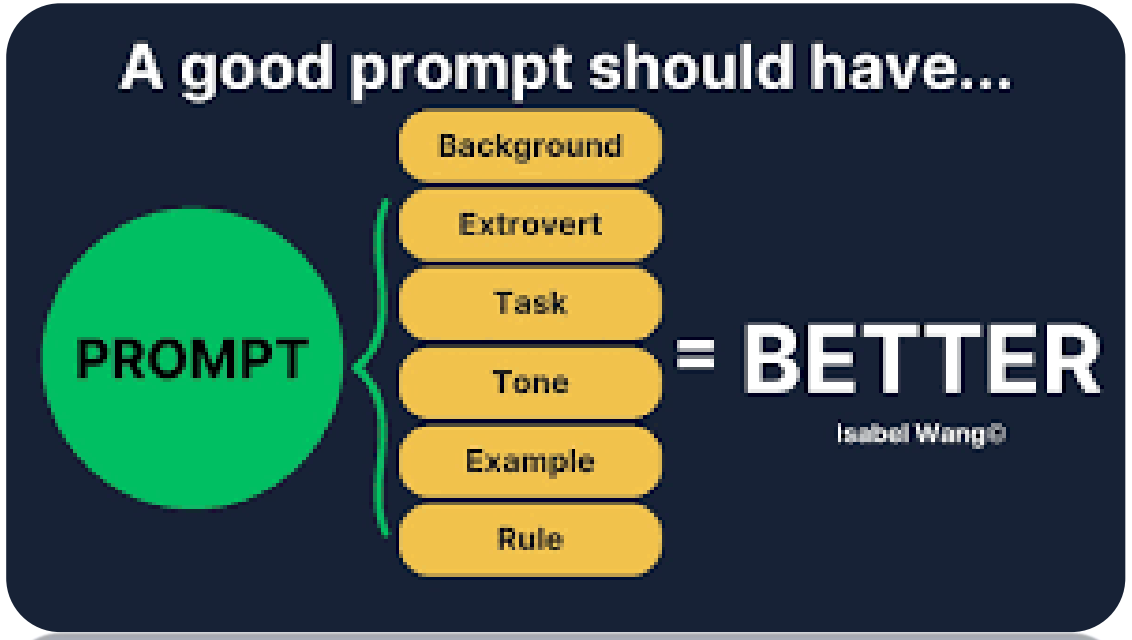
Learned

- Same model
- Different prompt
- Different outcome

SECTION 2: BASIC PROMPTING PATTERNS

Prompt Anatomy

- **A good prompt contains**:
  - Role
  - Task
  - Context
  - Constraints
  - Output format

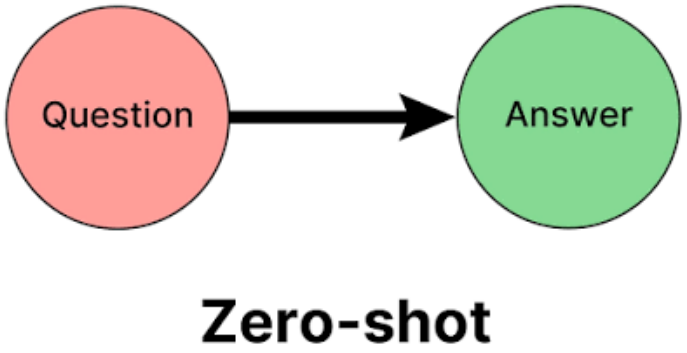


LAB 2B – Structured Prompting (Business Email Example)

- **Objective**
- Understand **instruction clarity & formatting control**

- **Prompt**
  - You are an HR manager.
  - Write a formal email informing employees about a new leave policy.
  - Limit response to 120 words.
  - Use bullet points.
  - Maintain professional tone.
- **Compare With**
  - Write an email about leave policy.

- **Key Learning**
  - Role + constraints improve quality
  - Business systems **cannot afford vague prompts**



SECTION 3: ZERO-SHOT PROMPTING

Zero-shot reasoning

- Zero-shot reasoning is the ability of a large language model to generate responses without any prior specific training. It relies entirely on what it already “knows” from the vast amount of data it was trained on.
- Zero-shot prompting involves posing a question or task to the model without providing any specific context or examples. To put it simply, it involves providing a single question or instruction to an LLM without any additional context or examples.
  - Most of us use generative AI tools like this, right?
- While it allows the model to generate responses based on its pre-existing knowledge, it can have limitations and lead to suboptimal results. That’s why it is always recommended to go through or cross-check the information provided by AI tools.

Zero-shot-CoT

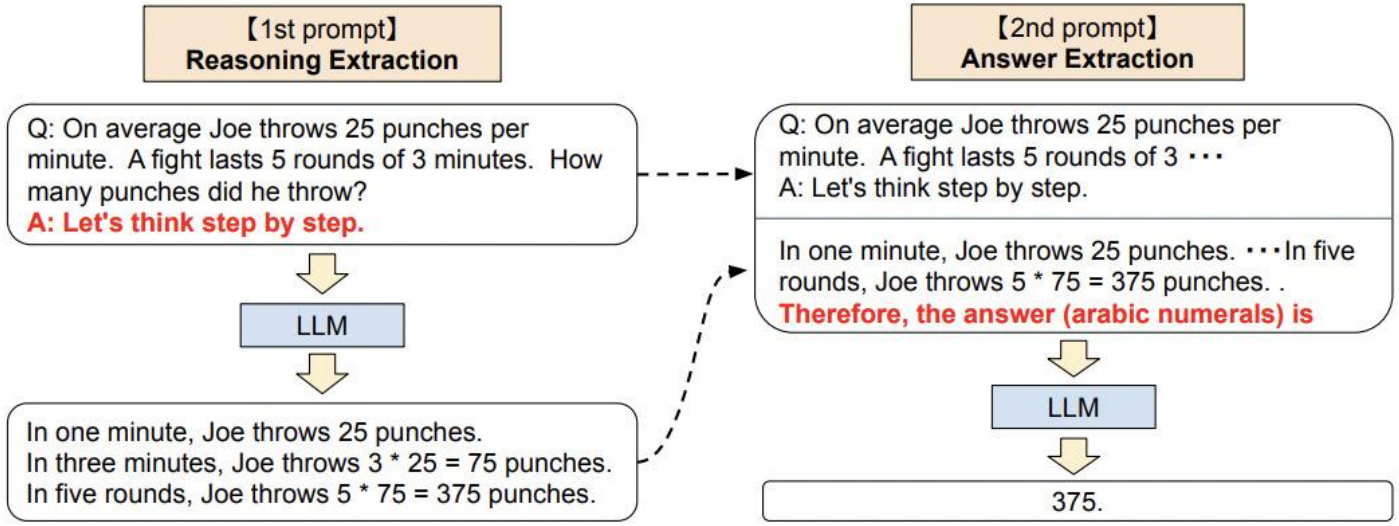


Figure 2: Full pipeline of Zero-shot-CoT as described in § 3: we first use the first "reasoning" prompt to extract a full reasoning path from a language model, and then use the second "answer" prompt to extract the answer in the correct format from the reasoning text.

Benefits of Zero-Shot

- Zero-shot reasoning is useful because it allows models to generate responses instantly without needing any extra training.
- This makes it fast, flexible, and cost-effective.
- It’s a great tool for quick insights, brainstorming, and exploring new topics without requiring pre-fed examples or specific datasets.
- Despite the many challenges of zero-shot reasoning, it provides a strong starting point for more complex tasks.

Zero-Shot Prompting

- No examples provided
- Model relies purely on instruction
- Fast, scalable
- Risky for complex tasks

Zero-Shot Limitations

- Reliance on Pre-existing Knowledge
- Lack of Contextual Understanding
- Limited Generalization
- Inaccurate or Incomplete Responses

Why to use Zero-shot prompting

INSTANT RESPONSES

No extra training or examples are needed, making it quick and efficient

FLEXIBILITY

It can handle a wide range of topics and questions without prior customization

Why Use Zero-Shot Prompting?

COST-EFFECTIVE

Since it doesn't require additional labeled data, it saves time and resources

EXPLORATORY INSIGHTS

Useful for brainstorming, getting general overviews, or testing an AI's baseline knowledge

datasciencedojo
data science for everyone

LAB 2C – Zero-Shot Classification (Business Routing)

- **Objective**
  - Route employee messages without training data.
  - Example text-
  - text = "Customer is unhappy with delayed refund and threatening escalation"
- **Business Insight**
  - No training required
  - Useful for **ticket routing, inbox automation**

```
classifier = pipeline(
    "zero-shot-classification",
    model="facebook/bart-large-mnli"
)
```

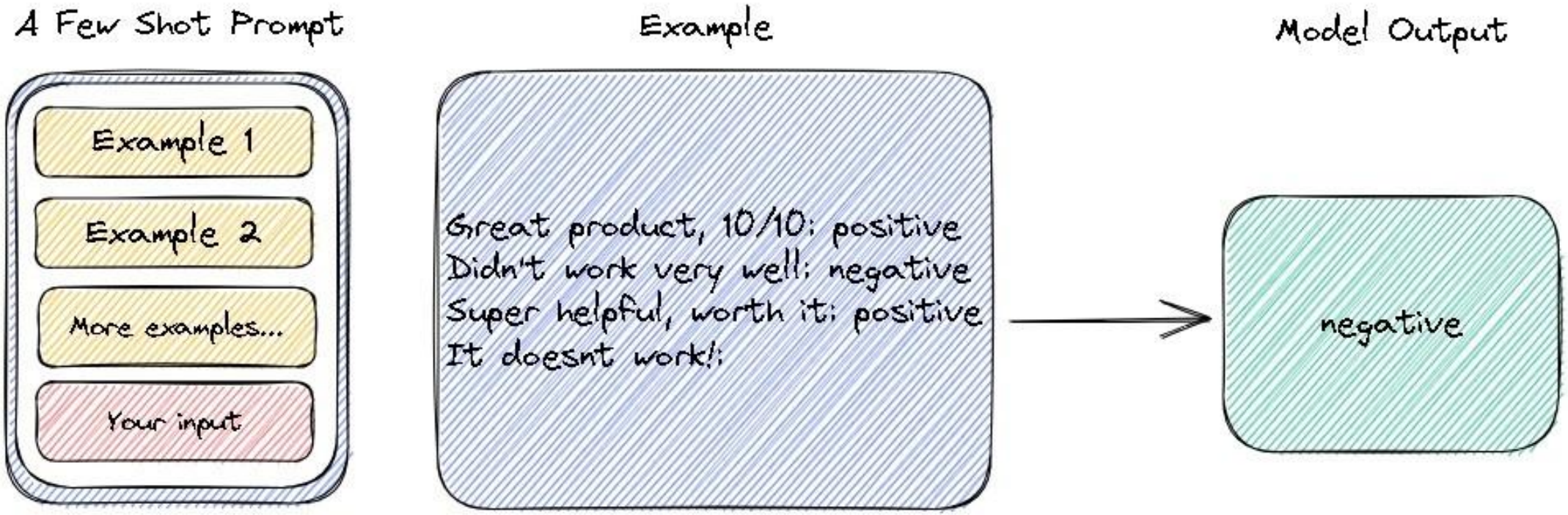
SECTION 4: FEW-SHOT PROMPTING

Few Shot Prompting

- Few-shot prompting is a prompt engineering technique where you provide a Large Language Model (LLM) with a small number of examples (typically **2 to 5**) to demonstrate a specific task, format, or tone.

- This approach leverages the model's ability for **in-context learning**, allowing it to adapt to new tasks in real-time without the need for additional training or fine-tuning.

Few-Shot Prompting



Key Differences in "Shot" Types

- **Zero-shot:** The model receives a direct instruction without any examples.
- **One-shot:** The model is given exactly one example to clarify expectations.
- **Few-shot:** The model receives multiple examples (usually 2–8) to recognize patterns and handle complex tasks.

Benefits and Use Cases

- **Structured Outputs:** Highly effective for teaching models to output data in specific formats like JSON, XML, or bulleted lists.
- **Specialized Domains:** Useful in legal, medical, or technical fields where a specific vocabulary or style is required but large datasets are unavailable.
- **Content Generation:** Helps match a specific brand voice or tone by providing previous successful content as "shots".
- **Improved Accuracy:** Often leads to more robust performance in reasoning tasks, such as sentiment analysis or complex code generation

Best Practices for Few-Shot Prompting

- **Use Diverse Examples:** Provide a range of examples that cover common use cases and potential edge cases.
- **Maintain Consistency:** Ensure all examples follow the same format (e.g., Input: [text] / Output: [label]) so the model can easily detect the pattern.
- **The "Last Shot" Rule:** Place your most important or representative example last, as models tend to emphasize the most recent information they read (recency bias).
- **Limit the Quantity:** Using more than 5–8 examples often results in diminishing returns and unnecessarily consumes "tokens," increasing costs

Limitations

- **Overfitting:** The model may mimic provided examples too closely and fail to generalize to new, unseen data.

- **Biases:** If examples are skewed (e.g., mostly positive sentiments), the model may develop a "majority label bias".

- **Token Consumption:** Long prompts with many examples increase processing time and operational costs.

Few-Shot Prompting

- Provide **examples**
- Reduces ambiguity
- Improves consistency
- Slightly higher token cost

LAB 2D – Few-Shot Sentiment Classification

- **Objective**
  - Improve accuracy using examples.
- **Classify the sentiment as Positive, Neutral, or Negative.**
- **Example:**
- Text: "The system works well"
- Sentiment: Positive
- **Example:**
- Text: "The update caused delays"
- Sentiment: Negative
- **Now classify:**
- Text: "The report is acceptable but needs changes"
- **Discussion**
  - Why was output more accurate?
  - When is few-shot better than zero-shot?

LAB 2D – Few-Shot Sentiment Classification

- **1. Priority & Escalation Detection (Operations / CX)**
- **Business Context**
- An operations team receives customer and internal messages.
  They must decide **urgency**, not destination.
- **Objective**
- Classify messages into:
  - High Priority (Immediate Action)
  - Medium Priority
  - Low Priority

LAB 2D1 – Few-Shot Sentiment Classification

- New Business Scenarios - **Change only the message**, not the examples.

- **Scenario 1**
- Message: "Multiple customers reporting failed transactions since morning"

- **Scenario 2**
- Message: "Laptop performance is slow after recent update"

- **Scenario 3**
- Message: "Monthly compliance report submission reminder"

LAB 2D2 – Few-Shot Sentiment Classification - Add Escalation Flag

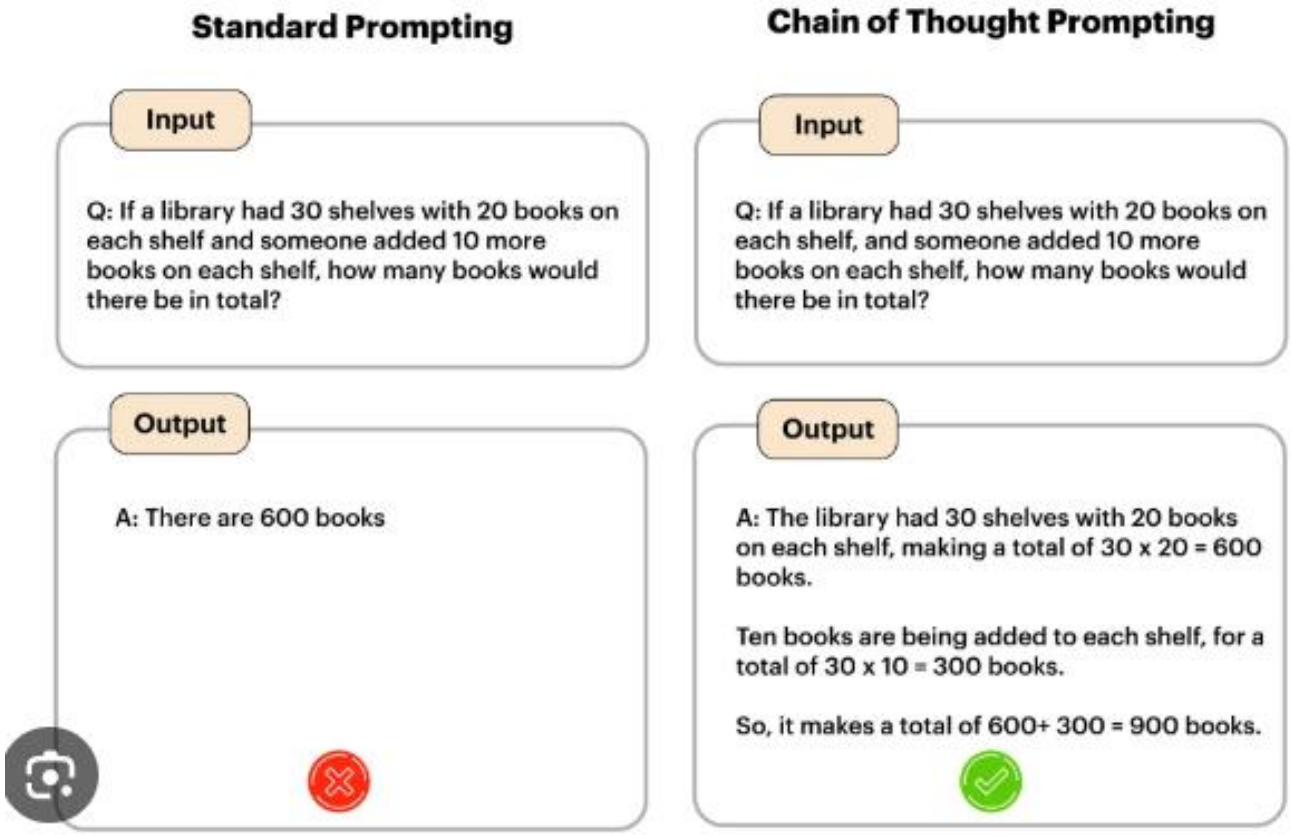
- **Updated Prompt**
- prompt = """
- You are an enterprise priority and escalation assessment system.
- Rules:
  - - High priority issues require immediate escalation.
  - - Medium priority issues require monitoring.
  - - Low priority issues do not require escalation.
- Examples:
  - Message: "Production server is down impacting customer transactions"
  - Priority: High
  - Escalation: Yes
  - Message: "Unable to access email for the last 2 hours"
  - Priority: Medium
  - Escalation: No
  - Message: "Requesting access to last quarter financial reports"
  - Priority: Low
  - Escalation: No
  - Now analyze:
  - Message: "Payment gateway intermittently failing during peak hours"
  - Priority:
  - Escalation:
  - """

SECTION 5: CHAIN OF THOUGHT (CoT)

Chain-of-thought (CoT)

- Chain-of-thought (CoT) prompting is an advanced prompt engineering technique that improves the reasoning capabilities of Large Language Models (LLMs) by guiding them to generate intermediate steps before reaching a final answer.
- Instead of jumping directly to a conclusion, the model "thinks out loud," which helps it solve complex multi-step problems in math, logic, and common-sense reasoning

EXAMPLE - CHAIN OF THOUGHT (CoT)



CORE VARIANTS - CHAIN OF THOUGHT

- **Zero-Shot CoT**: The simplest form, achieved by adding a phrase like **"Let's think step by step"** to the end of a prompt. This triggers the model to decompose the problem without needing specific examples.

- **Few-Shot CoT**: Provides the model with a few examples (exemplars) that demonstrate both the question and the step-by-step reasoning leading to the answer.

- **Automatic CoT (Auto-CoT)**: Minimizes manual effort by using the LLM itself to generate reasoning chains for various demonstration examples.

- **Multimodal CoT**: Extends reasoning to include non-textual inputs, such as images, allowing the model to analyze visual data step-by-step to reach a conclusion.

Key Benefits

- **Enhanced Accuracy**: By breaking down problems into smaller parts, the model is less likely to make logical or calculation errors.

- **Transparency**: Users can see the internal logic used to reach a result, making it easier to verify the answer or identify exactly where a reasoning error occurred.

- **Handling Complexity**: It enables LLMs to tackle tasks that standard prompting often fails at, such as complex symbolic manipulation or multi-hop question answering.

Best Practices for 2025

- **Model Size Matters**: CoT is an "emergent ability" that typically yields the best results on larger models (e.g., 100B+ parameters). Smaller models may generate "illogical" reasoning chains that actually decrease accuracy.

- **Avoid Over-Prompting**: For simple, direct-fact questions, CoT can be counterproductive, increasing latency and cost without improving the answer.

- **Combine with Other Techniques**: In 2025, CoT is frequently combined with Self-Consistency—running multiple reasoning paths and choosing the most frequent answer—to further boost reliability.

Chain of Thought Prompting

Encourages **step-by-step reasoning**

Critical for:

- Decision-making
- Policy interpretation
- Risk assessment

LAB 2E – Chain of Thought (Policy Reasoning)

- **Objective**
  - Show reasoning transparency.
- **Prompt**
  - An employee wants to take Privilege Leave from Friday to next Wednesday.
  - Company policy excludes weekends from PL calculation.
- **Think step by step and answer:**
  - How many PL days will be deducted?
- **Follow-up Prompt**
  - Answer directly without explanation.
- **Learning**
  - Reasoning prompt = safer decisions
  - Direct answers hide logic (risk for compliance)

LAB 2E – Chain of Thought (Policy Reasoning)

- **Business Context**
- An organization applies **Privilege Leave (PL)** based on policy rules.
  Incorrect calculation can lead to:
  - Payroll errors
  - Compliance violations
  - Employee disputes
- LLMs are increasingly used to **answer HR policy questions**.
- **Objective**
- Demonstrate:
  - How **reasoning prompts expose decision logic**
  - Why **direct answers without reasoning are risky**
- **Scenario**
  - Employee applies for **Privilege Leave**
  - Leave period: **Friday to next Wednesday**
- Company policy:
  Weekends (Saturday & Sunday) are excluded from PL calculation

LAB 2E – Chain of Thought (Policy Reasoning)

- **Why this fails**
  - **GPT-2 is NOT instruction-following**
  - **It does not reason reliably**
  - **It does not calculate deterministically**
  - It only *continues text probabilistically*
- So when you ask:
- "Think step by step and calculate…"
  - GPT-2:
  - Does **not actually calculate**
  - Just **generates text that looks like reasoning**
  - Output will vary and often be incorrect
- This is actually **GOOD for teaching**, because it proves:
  - LLMs ≠ calculators
    Reasoning ≠ guaranteed correctness

LAB 2E – Chain of Thought (Policy Reasoning)

- **2. Correct Lab Design (Industry-Grade Approach)**
- **What we SHOULD do**

Part	Responsibility
Policy reasoning	LLM
Date logic & leave calculation	Python
Transparency vs compliance risk	Prompting style

- **Correct Architecture for This Lab**
  - User Input → Python Calculation → LLM Explanation / Summary
  - This is **exactly how enterprise AI systems work**.

SECTION 6: TREE OF THOUGHTS (ToT)

Tree of Thoughts

- **Tree of Thoughts (ToT)** is an advanced AI prompting framework that enables Large Language Models (LLMs) to perform deliberate, non-linear reasoning.

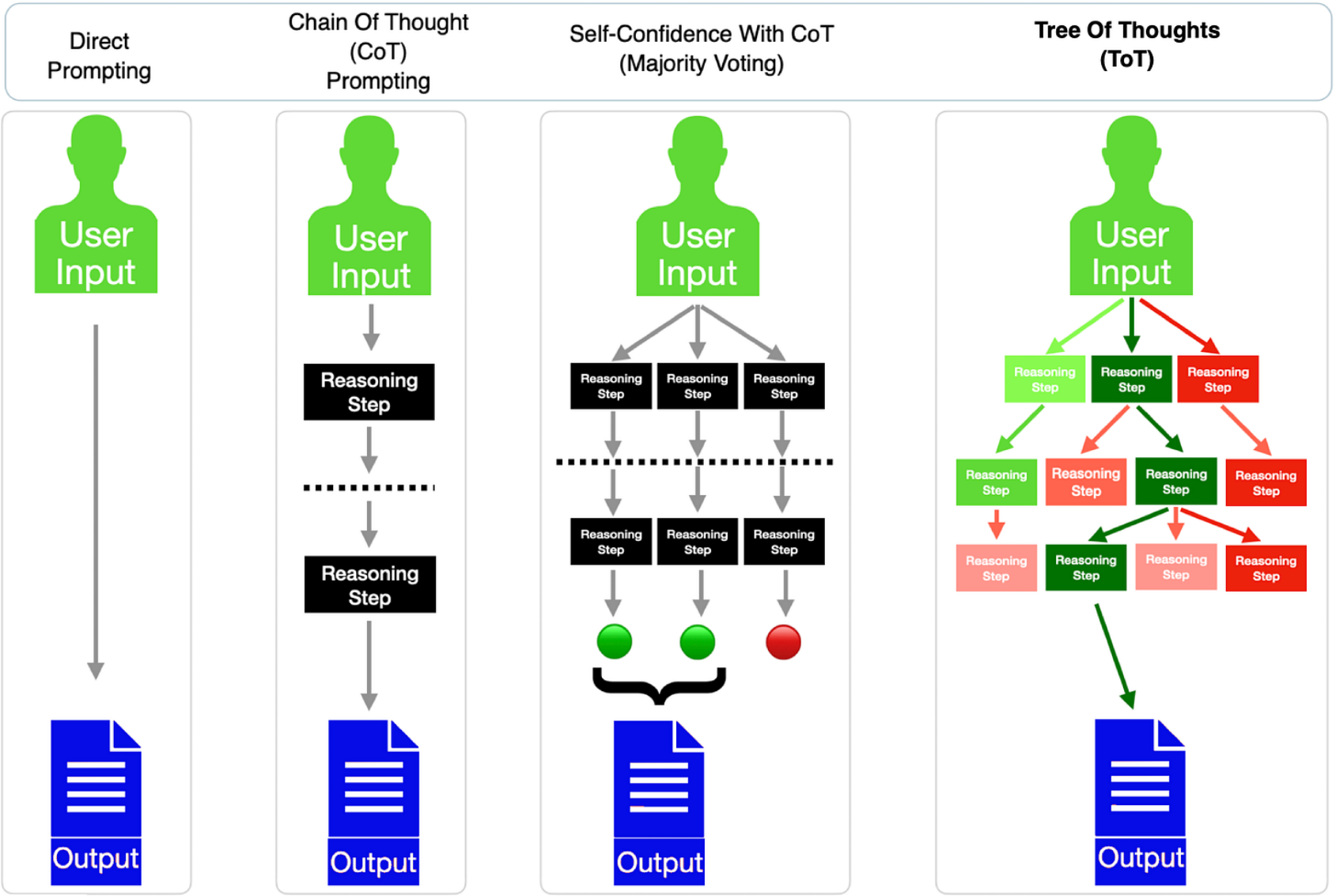
- Unlike traditional "Chain of Thought" (CoT) methods that follow a single path, ToT structures problem-solving as a search through a tree of potential intermediate steps

Tree of Thoughts - Core Mechanics

- The framework allows an LLM to explore multiple reasoning paths simultaneously, evaluate their progress, and backtrack if a path leads to a dead end. It relies on four essential components:
  - **Thought Decomposition:** Breaking a complex task into smaller, manageable "thought" steps.
  - **Thought Generation:** Generating multiple candidate thoughts for each step using techniques like sampling or sequential proposal.
  - **State Evaluation:** Using the LLM to score or vote on the quality of different reasoning branches to determine which are most promising.
  - **Search Algorithm:** Navigating the tree using standard algorithms like **Breadth-First Search (BFS)** or **Depth-First Search (DFS)** to find the optimal solution

Tree Of Thoughts Prompting (ToT)

With Implementation from LangChain



Adapted From: https://arxiv.org/pdf/2305.10601

Key Differences Between Chain of Thought and Tree of Thought:

Aspect	Chain of Thought (CoT)	Tree of Thought (ToT)
Structure	Linear progression (one path at a time)	Branching structure (multiple paths explored)
Exploration	One step follows the next sequentially	Parallel exploration of multiple possibilities
Efficiency	Efficient for simple, clear problems	More effective for complex, multi-step problems
Reasoning Style	Focuses on deep, single-path reasoning	Involves broad exploration of multiple paths
Decision Process	Premise leads directly to conclusion	Multiple branches evaluated to find optimal outcome
Use Case	Simple, deterministic problems	Complex, uncertain, or ambiguous problems

Practical Applications

- ToT is particularly effective for tasks requiring strategic planning or complex logic, such as:
  - **Creative Writing:** Generating and evaluating multiple plot outlines before drafting.
  - **Mathematical Reasoning:** Solving multi-step problems where initial errors derail the entire result.
  - **Business Operations:** Managing contract intake workflows by evaluating risk across separate reasoning paths.
  - **Software QA:** Identifying root causes of defects by exploring parallel diagnostic paths.

Tree of Thoughts

Explores **multiple reasoning paths**

Evaluates alternatives

Useful for **strategy & planning**

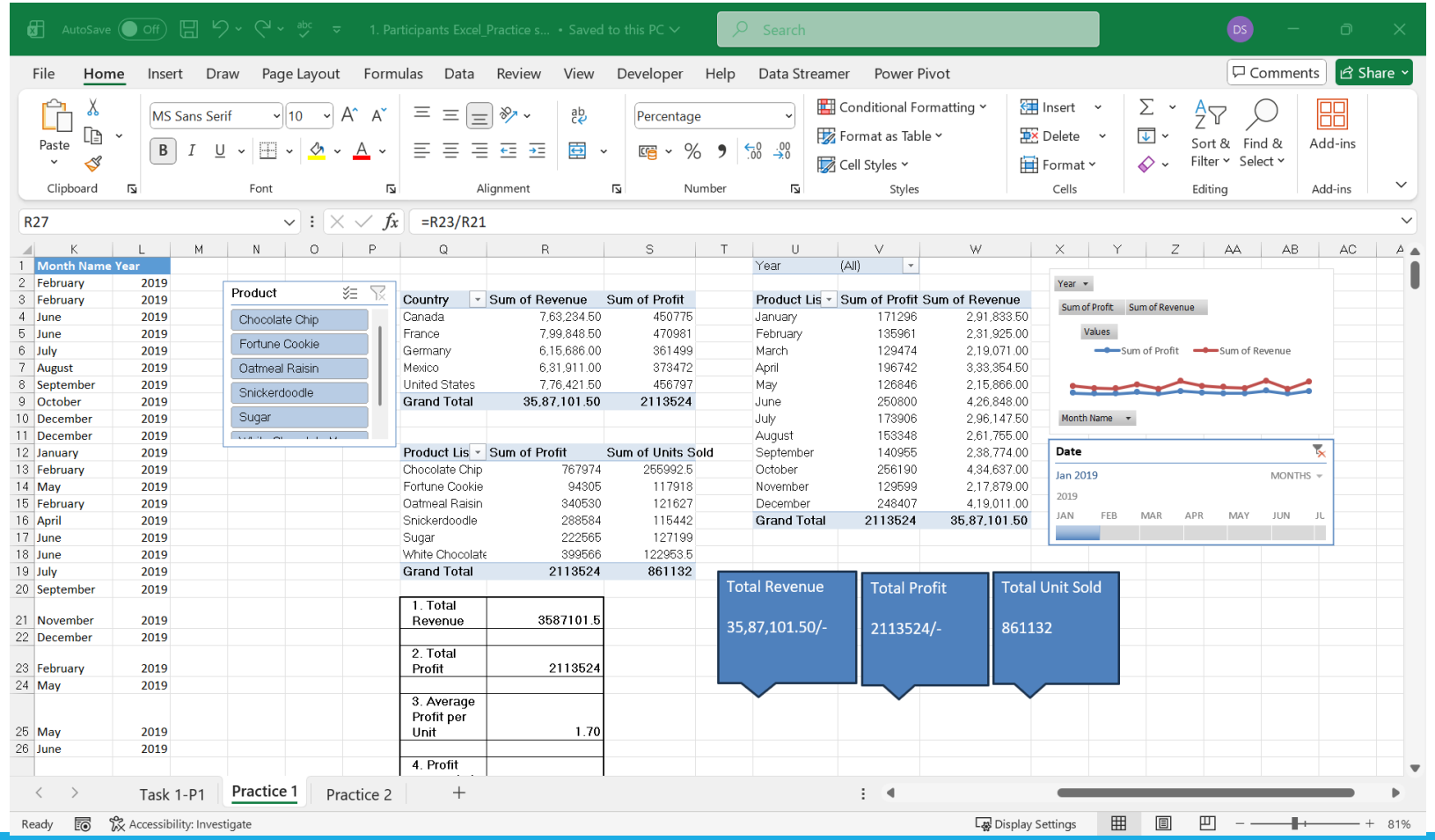
LAB 2F – TOT- Business Decision Exploration

- **Objective**
  - Simulate multiple decision paths.
- **Prompt**
  - We want to reduce customer churn.
  - Think of 3 different strategies.
  - For each strategy:
  - - Pros
  - - Cons
  - - Risk
  - Then recommend the best option.
- **Business Value**
  - Strategy brainstorming
  - Scenario planning
  - Management decision support

SECTION 7: LLM AS A JUDGE

DAY 3

Data Analysis – Cookies Business



Month Name	Year
February	2019
February	2019
June	2019
June	2019
July	2019
August	2019
September	2019
October	2019
December	2019
December	2019
January	2019
February	2019
May	2019
February	2019
April	2019
June	2019
June	2019
July	2019
September	2019
November	2019
December	2019
February	2019
May	2019
May	2019
June	2019

Product: Chocolate Chip, Fortune Cookie, Oatmeal Raisin, Snickerdoodle, Sugar

Country	Sum of Revenue	Sum of Profit
Canada	7,63,234.50	450775
France	7,99,848.50	470981
Germany	6,15,686.00	361499
Mexico	6,31,911.00	373472
United States	7,76,421.50	456797
Grand Total	35,87,101.50	2113524

Product Lis	Sum of Profit	Sum of Units Sold
Chocolate Chip	767974	255992.5
Fortune Cookie	94305	117918
Oatmeal Raisin	340530	121627
Snickerdoodle	288584	115442
Sugar	222565	127199
White Chocolate	399566	122953.5
Grand Total	2113524	861132

1. Total Revenue	3587101.5
2. Total Profit	2113524
3. Average Profit per Unit	1.70
4. Profit	

Year (All)

Product Lis	Sum of Profit	Sum of Revenue
January	171296	2,91,833.50
February	135961	2,31,925.00
March	129474	2,19,071.00
April	196742	3,33,354.50
May	126846	2,15,866.00
June	250800	4,26,848.00
July	173906	2,96,147.50
August	153348	2,61,755.00
September	140955	2,38,774.00
October	256190	4,34,637.00
November	129599	2,17,879.00
December	248407	4,19,011.00
Grand Total	2113524	35,87,101.50

Total Revenue 35,87,101.50/-

Total Profit 2113524/-

Total Unit Sold 861132

LLM as Evaluator

- LLM as Evaluator (or "LLM-as-a-judge") uses one large language model to assess the quality and performance of another LLM's outputs, acting like a human judge with a defined rubric in a prompt.

- This method offers scalability, flexibility, and speed for tasks like checking for bias, hallucinations, relevance , or adherence to tone, surpassing traditional metrics by capturing nuance.

- It's crucial for quality assurance, monitoring, and continuous improvement in AI applications, often combined with human feedback for best results.

How it Works

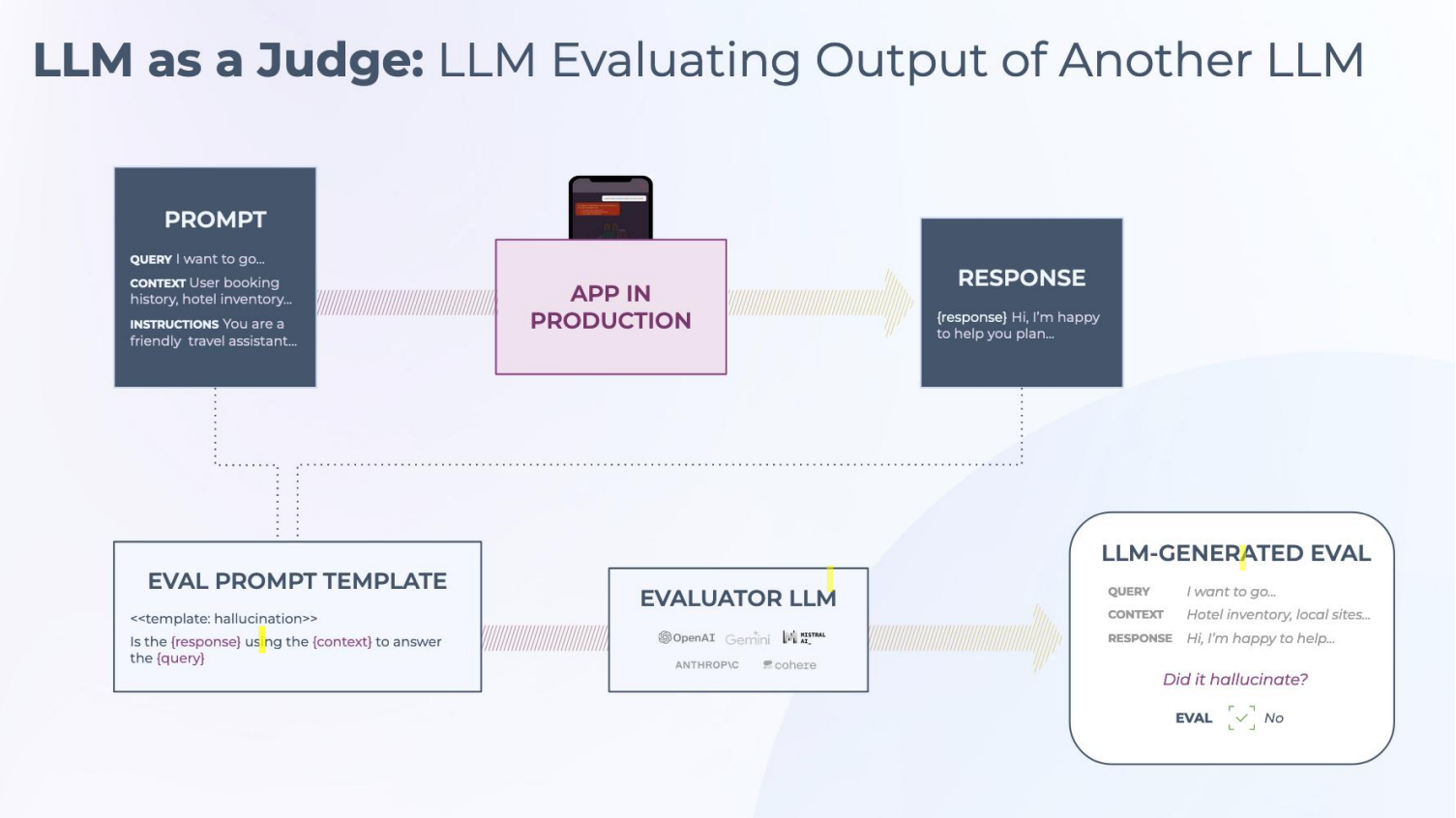
- **Define Criteria**: Create a prompt detailing evaluation rules (e.g., politeness, factuality, helpfulness).

- **Input Data**: Provide the LLM evaluator with the original prompt, the generated response, and potentially reference materials (for factual checks).

- **Judge & Score**: The LLM "judges" the output, assigning scores or classifications (e.g., "Good," "Bad," or a numerical rating) with reasoning (chain-of-thought).

- **Output**: Receive structured feedback, scores, and rationales for each evaluation.

LLM as Evaluator



Key Benefits

- **Scalability**: Evaluate thousands of outputs quickly, much faster than humans.
- **Nuance**: Assesses subjective qualities (tone, naturalness) better than simple metrics like BLEU/ROUGE.
- **Flexibility**: Easily adjust criteria by changing the prompt.
- **Consistency**: Provides repeatable evaluations with a fixed rubric.

LLM as Evaluator

LLM evaluates other outputs

Used in:

- QA
- Model evaluation
- Resume screening checks

LAB 2G – LLM as Judge (Business Decision Review)

- **Scenario**

  A bank's ML model flagged a transaction as "High Risk".
  - Two explanations are generated:
  - Explanation A (short, vague)
  - Explanation B (detailed, policy-aligned)

- **Task**

  Use an LLM to judge:
  - Which explanation is better
  - Why
  - Risk level of each explanation

- **Key Learning**
  - LLMs can **evaluate reasoning**, not just generate text.

SECTION 8: PROMPT FAILURES & HALLUCINATIONS

Prompt failures and hallucinations in AI

- Prompt failures and hallucinations in AI happen when a user's input (prompt) is unclear, contradictory, or too complex, causing the Large Language Model (LLM) to generate believable but false, fabricated, or irrelevant information, essentially "making things up" because it's predicting patterns, not verifying facts, leading to errors like fake citations, wrong facts, or illogical outputs that require careful prompt engineering and grounding in real data (RAG) to fix.

What are Prompt Failures?

- **Ambiguous/Vague Prompts:** Unclear intent pushes the model to guess, leading to off-topic or imaginative content.
- **Conflicting Prompts:** Inconsistent instructions confuse the AI, forcing it to reconcile contradictions, resulting in illogical responses.
- **Adversarial Prompts:** Deliberately crafted inputs designed to trick the model into producing errors.
- **Misleading Prompts:** Prompts with slang, idioms, or complexity the model hasn't learned well, causing nonsensical outputs.

What are Hallucinations?

- **Definition:** Plausible-sounding but factually incorrect, misleading, or entirely fabricated information generated by an AI.
- **Causes:** Poor prompts, outdated/insufficient training data, errors in retrieving external data, overfitting, or the model's core function of predicting likely text sequences.
- **Examples:** Inventing fake citations, incorrect facts, non-existent people, or impossible physics (like a hand grabbing a banana in a video).
- **Types:** Sentence contradictions, prompt contradictions (conflicts with input), factual inaccuracies, or irrelevant info.

How to Fix Them (Solutions)

- **Better Prompting:** Be specific, provide context, use clear instructions, and add fallback rules (e.g., "only answer if you know").
- **Retrieval-Augmented Generation (RAG):** Connect the AI to verified external data sources to ground answers in facts.
- **Model Fine-Tuning:** Train models on high-quality, domain-specific data to improve accuracy in specialized areas.
- **Step-by-Step Reasoning:** Ask the model to show its work to make outputs more transparent and verifiable.
- **Guardrails:** Implement input/output filters to catch errors before they reach the user.

Why Prompts Fail

- Ambiguous instructions
- Missing constraints
- Overloaded context
- Asking for unknown facts

LAB 2H – Hallucination Demonstration

- **Objective**
  - Show model confidence even when wrong.
    - Hallucination
    - Overconfidence
    - LLM-as-a-Judge motivation
    - Enterprise risk
- **Prompt**
  - How US politics affect the Indian Corporates?
  - What are our company helpdesk policies.
- **Discussion**
  - Did model answer confidently?
  - Why this is dangerous in enterprise systems?

SECTION 9: BUSINESS PROMPTING BEST PRACTICES

Enterprise Prompt Guidelines

- Always define role
- Always define boundaries
- Avoid open-ended questions
- Use validation prompts
- Log prompts (audit trail)

FINAL LAB – Prompt Engineering Challenge

- **Scenario**
  - You are designing an **AI assistant for a Bank's Internal Operations Portal**.

- The assistant will be used by:
  - Branch managers
  - Operations staff
  - Risk analysts

- The system must respond to:
  - Operational queries
  - Policy-related questions
  - Escalation scenarios

- ⚠️ **Incorrect answers may lead to regulatory violations.**

FINAL LAB – Prompt Engineering Challenge

- **TASK SET 1 – SAFE POLICY ASSISTANT (STRICT MODE)**
- **Objective**
- Design a prompt that ensures:
  - **Formal tone**
  - **Policy-bounded responses**
  - **No guessing**
  - **Clear refusal when data is missing**
- **Prompt Design Task**
- **Participants must write a SYSTEM PROMPT that ensures:**
- The assistant should:
  - Answer **only** using provided policy text
  - Clearly say *"Information not available"* if policy is missing
  - Avoid assumptions or inferred rules
  - Maintain formal, compliance-appropriate language
  - Suggest escalation when unsure
- **Example Query :** "Can a branch override transaction limits for VIP customers?"
- **Expected Deliverable**
- **Deliverable 1**
- Prompt text (system instruction)
- **Deliverable 2**
- Explanation:
  - Why guardrails were added
  - How hallucination is prevented
  - Why tone matters in compliance

FINAL LAB – Prompt Engineering Challenge

- **Deliverable 1: Prompt Text (System Instruction)**
- Lab includes a clear **system instruction (prompt)**:
  - You are a Banking Compliance AI Assistant.
- Rules:
  - 1. Answer ONLY using the provided policy text.
  - 2. Do NOT infer, assume, or guess missing information.
  - 3. If the answer is not explicitly available, say:
  - "The requested information is not available in the provided policy."
  - 4. Maintain a formal and compliance-approved tone.
  - 5. If risk exists, advise escalation to the compliance team.
  - 6. Never sound confident when information is missing.
- **Why this qualifies:**
  - Acts as a **hard guardrail**
  - Overrides user intent if it violates policy
  - Controls **behavior, tone, and boundaries**
  - Meets enterprise AI governance expectations

1. Why Guardrails Were Added

- Guardrails were added to:
  - Prevent unauthorized policy interpretation
  - Ensure the AI does not exceed its mandate
  - Align AI behavior with **banking compliance standards**
- Reduce operational and regulatory risk
- In regulated environments, **AI must not act as a decision-maker**. Guardrails enforce the AI's role as an **information assistant only**.

2. How Hallucination Is Prevented

- **Covered — and this is a strong point of your lab**
- Hallucination is prevented by:
- This ensures:
  - No fabricated answers
  - No confident misinformation
  - Predictable, auditable outputs

Mechanism	**Purpose**
“Answer ONLY using the provided policy text”	Blocks external knowledge
Explicit “do NOT infer or guess” rule	Stops assumption
Mandatory fallback response	Ensures safe failure
Temperature = 0.0	Eliminates creative variation

3. Why Tone Matters in Compliance

- **Covered — but must be verbalized clearly**
- In compliance scenarios:
  - Overconfidence = **legal and reputational risk**
  - Casual tone = **misinterpretation**
  - Friendly language = **false authority**
- Your system prompt enforces:
  - Formal language
  - Neutral phrasing
  - Escalation instead of advice
- This mirrors real-world compliance communications:
  - Audit teams
  - Risk offices
  - Regulatory responses

FINAL LAB – Prompt Engineering Challenge

- TASK SET 2 – OPERATIONAL GUIDANCE ASSISTANT (ADVISORY MODE)
- **Objective**
- Design a second prompt for:
  - Day-to-day operational support
  - Still safe, but more **guidance-oriented**
  - Allows suggestions but with disclaimers
- **Prompt Design Task**
  - The assistant should:
  - Provide **procedural guidance**
- Clearly separate:
  - Facts vs recommendations
- Include disclaimers:
  - "Please confirm with compliance team"
  - Maintain professional but approachable tone
- **Example Query**
- "What steps should I follow if a customer disputes a transaction?"
- **Deliverables**
- **Deliverable 3**
  - Advisory prompt text
- **Deliverable 4**

  Explanation:
  - How this differs from strict mode
  - Why enterprises need multiple prompt personas

FINAL LAB – Prompt Engineering Challenge

- TASK SET 3 – PROMPT FAILURE ANALYSIS (CRITICAL THINKING)

- **Objective**
- Force reflection, not just writing prompts.
-

- **Challenge Questions**

- Participants must answer:
- What could go wrong if:
  - Advisory prompt is used for compliance queries?
- What business risks arise if:
  - Model sounds confident but policy is missing?
- Should these prompts be:
  - User-editable?
  - System-locked?

Key Learnings of the day

- Design prompts intentionally
- Understand prompt risks
- Apply advanced prompting in business contexts

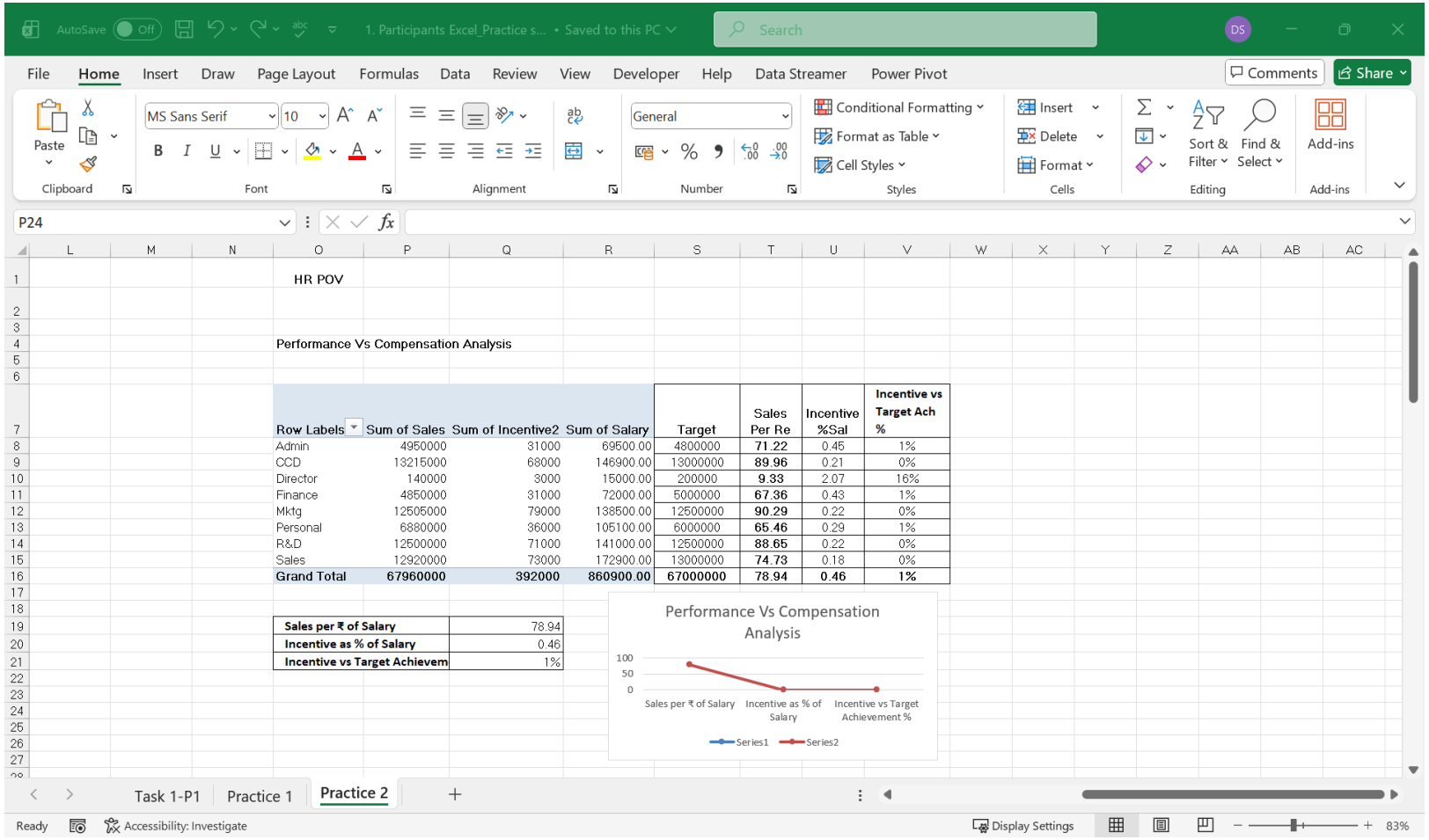
Prompt Engineering (Advanced) & LLM Reasoning Control

DAY 4-5

Why Prompt Engineering Matters (Enterprise View)

- Applications of LLMs in Enterprise Systems
- LLMs as Decision Support (Not Decision Makers)
- Ethics of AI – Practical Enterprise View
- Applied Labs
- Week 4 Final Assessment

Data Analysis – HR & Sales Analysis



HR POV

Performance Vs Compensation Analysis

Row Labels	Sum of Sales	Sum of Incentive2	Sum of Salary	Target	Sales Per Re	Incentive %Sal	Incentive vs Target Ach %
Admin	4950000	31000	69500.00	4800000	71.22	0.45	1%
CCD	13215000	68000	146900.00	13000000	89.96	0.21	0%
Director	140000	3000	15000.00	200000	9.33	2.07	16%
Finance	4850000	31000	72000.00	5000000	67.36	0.43	1%
Mktg	12505000	79000	138500.00	12500000	90.29	0.22	0%
Personal	6880000	36000	105100.00	6000000	65.46	0.29	1%
R&D	12500000	71000	141000.00	12500000	88.65	0.22	0%
Sales	12920000	73000	172900.00	13000000	74.73	0.18	0%
Grand Total	67960000	392000	860900.00	67000000	78.94	0.46	1%

Sales per ₹ of Salary	78.94
Incentive as % of Salary	0.46
Incentive vs Target Achievem	1%

Recap

What we have already covered

- Prompt engineering (zero, few, CoT)
- Prompt personas (strict vs advisory)
- LLM hallucinations
- LLM as Judge
- Compliance-safe prompting

SECTION 1: APPLICATIONS OF LLMs

(ENTERPRISE LENS)

Why LLM Applications Matter

- **Key Idea**
  LLMs are not chatbots.
  They are **capabilities embedded inside systems**.

- **LLMs are used to**
  - Interpret
  - Summarize
  - Reason
  - Assist decisions
  - Support humans

- **Not to**
  - Take final decisions
  - Replace accountability

Enterprise Application Categories

Domain	Example Use Case
Customer Operations	Dispute resolution assistant
Banking Ops	Transaction explanation
Risk & Compliance	Policy interpretation
Finance	Credit memo summarization
IT Ops	Incident triage
Knowledge Mgmt	Semantic search on SOPs

Application Architecture Pattern

- **Typical Enterprise LLM Flow**

User Query

↓

Prompt Persona (Strict / Advisory / Friendly)

↓

LLM

↓

Guardrails (Policy, Tone, Risk)

↓

Human Review / Action

- **Key Insight**

LLM output is **never the final step**.

LLM as Decision Support vs Decision Maker

Aspect	Decision Support	Decision Maker
Responsibility	Human	AI
Risk	Controlled	High
Compliance	Auditable	Dangerous
Enterprise Fit	✅	❌

Enterprises deploy LLMs as *copilots*, not pilots.

LAB 1: Application Mapping Exercise

- **Objective**
  Think like a **solution architect**, not a prompt writer.

- **Scenario**
  A BFSI organization wants to deploy LLMs internally.

LAB 1: Application Mapping Exercise

- LAB 1: TASK –
- Define **ONE LLM use case** and fill below table:

Item	**Answer**
Business Function	CRM, Ops, HR, MKTn., Finance
End User	Customer/Client, Emp., Vendors/Suppliers(Third parties), Govt. – Citizens
Problem Solved	Cust. Queries – Bot, Translation, Search, Calls, Email
Prompt Persona	Strict, Advisory, Friendly
Risk Level (Low/Med/High)	YES
Human Approval Needed?	YES

SECTION 2: ETHICS OF AI

(PRACTICAL & ENTERPRISE-DRIVEN)

Why Ethics Matters for LLMs

- **Key Reality**

- LLMs:
  - Sound confident
  - Can be wrong
  - Influence decisions
  - **Ethics ≠ Philosophy**

Ethics = **Risk Management**

Core Ethical Risks in LLM Systems

Risk	What It Looks Like
Hallucination	Confident wrong answers
Bias	Unequal responses
Overconfidence	No disclaimers
Data Leakage	Sensitive info in prompts
Automation Bias	Blind trust by users

Ethics Controls (Enterprise View)

Risk	**Control Mechanism**
Hallucination	“Say I don’t know” rules
Bias	Prompt constraints
Legal Risk	Strict personas
Trust Risk	Disclaimers
Audit Risk	Logging & reviews

🔬 LAB 2 – ETHICAL FAILURE ANALYSIS

LAB 2: Ethical Failure Scenario

- **Scenario**

An AI assistant:

- Answered a compliance query confidently
- The answer was wrong
- Customer suffered financial loss

- **DISCUSSION QUESTIONS**

- Where did the failure occur?
- Which guardrail was missing?
- Prompt issue or system issue?
- Who is accountable?
- How would you redesign the system?

SECTION 3: LLMs IN REAL ENTERPRISE SYSTEMS

Why Enterprises Need Multiple Prompt Personas

- Single AI ≠ Single Behavior

Persona	Purpose
Strict	Compliance & policy
Advisory	Guidance with caution
Friendly	UX & engagement

- **Insight**
  Different risk → different prompt.

When to Use Which Persona

Use Case	**Persona**
Policy interpretation	
Operational guidance	
FAQ / Helpdesk	
Legal / Audit	

When to Use Which Persona

Use Case	Persona
Policy interpretation	Strict
Operational guidance	Advisory
FAQ / Helpdesk	Friendly
Legal / Audit	Strict only

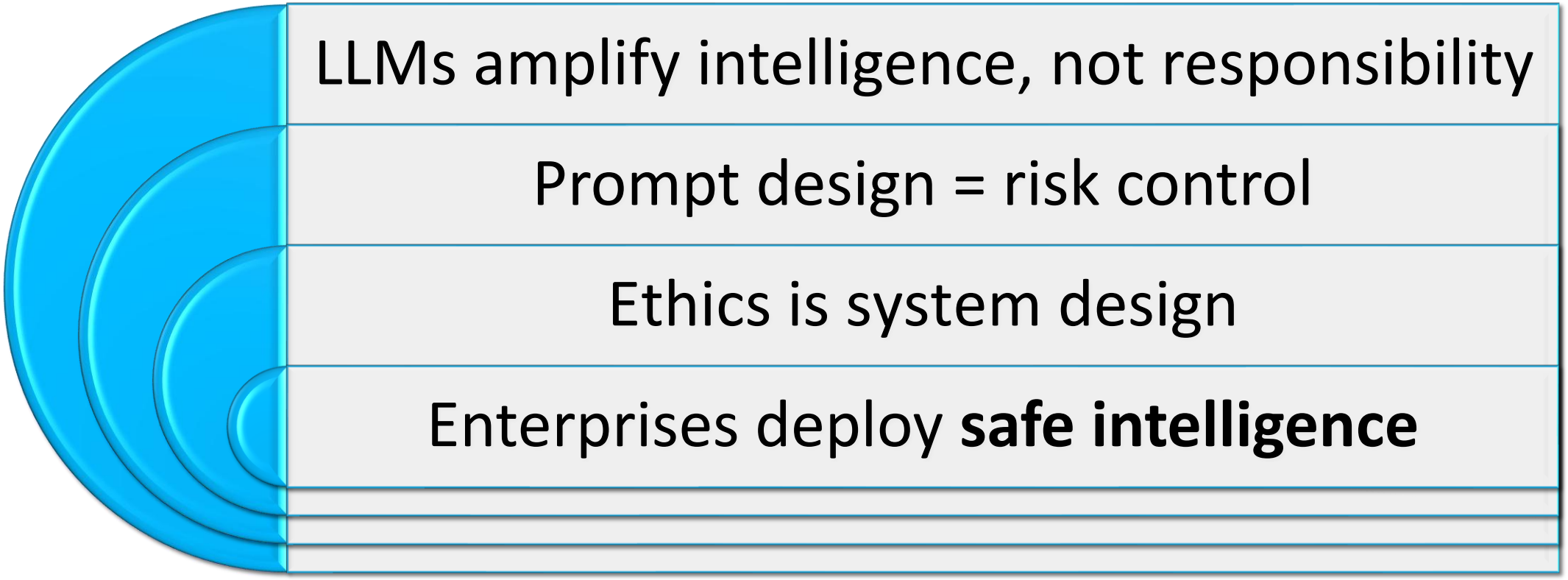
🔬 LAB 3 – DECISION SUPPORT COMPARISON : Output Comparison

- Given **three AI responses**:
  - Confident, no disclaimer
  - Advisory with disclaimer
  - Escalation-only

- Teams decide:
  - Which is safest?
  - Which is usable?
  - Which would enterprise deploy?

- **Key Learning**
  - Best answer ≠ safest answer

Key Takeaways



WEEK 4 - FINAL ASSESSMENT

STRUCTURE

PART A: 25 MCQS

PART B: 1 BUSINESS CASE

Part 2 – ASSESSMENT - BUSINESS CASE

- **Scenario**

  A banking organization wants to deploy AI for:
  - Customer disputes
  - Policy interpretation
  - Operations support

- **BUSINESS CASE TASKS**
  - Identify 3 LLM use cases
  - Choose prompt persona for each
  - Identify ethical risks
  - Propose guardrails
  - Decide where human approval is mandatory

What we learned till now

- Control LLM outputs using **prompt structure**
- Apply **zero-shot, few-shot, chain-of-thought**
- Compare **reasoned vs direct answers**
- Use LLMs safely for **policy, business & compliance use-cases**
- Understand **why hallucination happens** and how to reduce it

Thank You!